Coach Feature — Implementation Plan & Automated Tests

Overview

This document covers the complete build sequence for the coach feature, every file that changes, and the automated health check journeys that validate each risk point. Tests are written to slot directly into `AppHealthCheck.jsx` using the existing `runJourney` / step pattern already in production.

Risk register

Risk	File	Severity	Mitigation
stripeWebhook silently breaks for paying parents	`stripeWebhook/entry.ts`	🔴 High	Wrap coach logic in isolated try/catch; test entitlement still created without coach code
useSeasonAccess gates coach accounts out	`useSeasonAccess` hook	🔴 High	Add role check before entitlement check; test both paths
Coach code stored in localStorage lost before checkout	`Checkout.jsx`	🟡 Medium	Verify code survives page navigation; test read at checkout time
Parent linked to wrong coach via stale code	`stripeWebhook`	🟡 Medium	Validate invite_code lookup before creating CoachRoster row
Coach sees demo/unpaid athlete data	`CoachDashboard`	🟡 Medium	Filter CoachRoster query to active entitlements only
Parent sees empty coach messages section	`Workspace.jsx`	🟢 Low	Conditional render — section only mounts when messages exist
Email notification fires for every message send	`sendCoachMessage` function	🟢 Low	Wrap in try/catch — messaging works even if email fails

---

Build phases

Phase 1 — Role infrastructure (touches existing files, deploy first)

Goal: Make the app safe to handle coach accounts before any coach accounts exist. No user-visible changes. Deploy and verify with health check before proceeding.

Files changed

`useSeasonAccess` hook

- Add `role === "coach"` bypass before the entitlement check
- Coach accounts return `{ hasAccess: false, isCoach: true, role: "coach" }` — no entitlement needed
- Null/missing role continues to default to parent behavior (no change for existing users)

`AuthRedirect.jsx`

- Add one branch: `if (role === "coach") nav("/CoachDashboard", { replace: true })`
- Sits above the existing entitlement poll logic — coaches never hit that path

`src/components/auth/CoachRoute.jsx` ← new file, mirrors AdminRoute.jsx

```jsx
// Checks role === "coach" — redirects to /Workspace if denied
// Same pattern as AdminRoute, swap email check for role check
export default function CoachRoute({ children }) {
  const { role, isLoading } = useSeasonAccess();
  if (isLoading) return null;
  if (role !== "coach") return <Navigate to="/Workspace" replace />;
  return children;
}
```

`pages.config.js`

- Add imports for `CoachDashboard`, `CoachSignup`, `CoachNetworkAdmin`
- Add routes to PAGES object
- No mainPage change

Health check journeys for Phase 1

javascript

```
// Add to ALL_JOURNEYS in AppHealthCheck.jsx under a new "Coach Feature" label

{
  label: "Coach Feature",
  journeys: [

    {
      id: "coach_role_bypass",
      name: "Coach role — useSeasonAccess bypass",
      icon: "🏈",
      description: "Accounts with role=coach are not gated by entitlement check. " +
        "Simulates the access shape a coach account would return.",
      steps: [
        {
          name: "useSeasonAccess hook is importable",
          run: async () => {
            // Verify the hook exists — indirect check via season context
            const season = window.__seasonAccessDebug || null;
            // We can't call the hook directly from a test, so we verify
            // the role field is present on the auth response
            const me = await base44.auth.me();
            if (!me) throw new Error("auth.me() returned null — cannot verify role f
            // role may be undefined on existing accounts — that is correct (default
            return `auth.me() ok — role field: ${me.role ?? "(not set, defaults to p
          },
        },
        {
          name: "Admin account has role=admin on auth object",
          run: async () => {
            const me = await base44.auth.me();
            if (!me) throw new Error("auth.me() returned null");
            if (me.role !== "admin") throw new Error(
              `Expected role=admin for admin account, got: ${me.role ?? "(undefined)
              "— role field may not be set on auth user object yet"
            );
            return `role=admin confirmed on admin account ✓`;
          },
        },
        {
          name: "CoachRoute component file exists in codebase",
          run: async () => {
            // Indirect: verify the route /CoachDashboard is registered in pages
            // by checking the PAGES config — if it's missing, navigation will 404
```

```
        const allRoutes = Object.keys(window.__pagesDebug || {});
        if (allRoutes.length === 0) {
          // Can't introspect pages at runtime — verify by entity probe instead
          return "Cannot introspect pages at runtime — verify CoachRoute.jsx exi
        }
        if (!allRoutes.includes("CoachDashboard")) {
          throw new Error("CoachDashboard not found in pages config — add to pag
        }
        return `CoachDashboard route registered ✓`;
      },
    },
  ],
},

{
  id: "coach_entity_schema",
  name: "Coach entity — schema and read access",
  icon: "📋",
  description: "Coach entity exists, is queryable, and has required fields. " +
    "Runs after Phase 2 deployment.",
  steps: [
    {
      name: "Coach entity is queryable",
      run: async (ctx) => {
        let coaches;
        try {
          coaches = await base44.entities.Coach.filter({});
        } catch (err) {
          throw new Error(
            `Coach entity not readable: ${err?.message || err} — ` +
            "create the Coach entity in base44 admin before deploying Phase 2"
          );
        }
        if (!Array.isArray(coaches)) throw new Error("Coach.filter() returned no
        ctx.coaches = coaches;
        return `Coach entity reachable — ${coaches.length} records`;
      },
    },
    {
      name: "Coach entity has required fields",
      run: async (ctx) => {
        if (ctx.coaches.length === 0) {
          return "No Coach records yet — field check skipped (expected before fi
        }
```

```
            const required = ["name", "school_or_org", "invite_code", "account_id",
            const sample = ctx.coaches[0];
            const missing = required.filter(f => !(f in sample));
            if (missing.length > 0) {
              throw new Error(`Coach record missing fields: ${missing.join(", ")} —
            }
            return `All required fields present on Coach entity ✓`;
          },
        },
        {
          name: "invite_code values are unique",
          run: async (ctx) => {
            if (ctx.coaches.length < 2) return "Fewer than 2 coaches — uniqueness ch
            const codes = ctx.coaches.map(c => c.invite_code).filter(Boolean);
            const unique = new Set(codes);
            if (unique.size !== codes.length) {
              throw new Error(
                `Duplicate invite_codes detected (${codes.length} codes, ${unique.si
                "invite_code must be unique across all Coach records"
              );
            }
            return `All ${codes.length} invite_codes are unique ✓`;
          },
        },
      ],
    },

  ],
},
```

Phase 2 — Coach signup + invite link

Goal: Coaches can create a free account and generate an invite link. Parents who click the link get the coach code stored silently.

Files changed

`src/pages/CoachSignup.jsx` ← new file

- Form: name, school/org, sport, email, password
- On submit: `base44.auth.register(email, password)` then set `role: "coach"` on account

- Create `Coach` entity record with generated `invite_code`
- Invite code format: `{LASTNAME}-{SCHOOLABBR}-{4-digit-random}` e.g. `SMITH-WHS-4821`
- Redirect to `/CoachDashboard` on success

`src/pages/Home.jsx`

- Add one line below existing CTAs: `"Are you a coach or trainer? →"` link to `/CoachSignup`
- No other changes to home page

`src/pages/CoachInviteLanding.jsx` ← new file, handles `/join?coach=CODE`

- Reads `coach` param from URL
- Stores in `localStorage` as `coachInviteCode`
- Immediately redirects to `/Home` — parent sees normal site, demo works as always
- If code is invalid (not found in Coach entity), stores nothing and redirects cleanly

`src/pages/Checkout.jsx`

- At checkout session creation, read `localStorage.getItem("coachInviteCode")`
- Pass as `coach_invite_code` in Stripe session metadata
- No other changes to checkout flow

Health check journeys for Phase 2

```javascript
```

```
{
  id: "coach_invite_code_flow",
  name: "Coach invite code — localStorage persistence",
  icon: "🔗",
  description: "Coach invite code survives being stored in localStorage and is " +
    "readable at checkout time. Does not require an actual coach account.",
  steps: [
    {
      name: "localStorage read/write available",
      run: async () => {
        try {
          localStorage.setItem("__hc_test__", "1");
          const val = localStorage.getItem("__hc_test__");
          localStorage.removeItem("__hc_test__");
          if (val !== "1") throw new Error("localStorage write/read mismatch");
          return "localStorage available ✓";
        } catch (err) {
          throw new Error(`localStorage not available: ${err.message} — coach invite
        }
      },
    },
    {
      name: "Simulate invite code store and retrieve",
      run: async (ctx) => {
        const testCode = "SMITH-WHS-TEST";
        localStorage.setItem("coachInviteCode", testCode);
        const retrieved = localStorage.getItem("coachInviteCode");
        localStorage.removeItem("coachInviteCode");
        if (retrieved !== testCode) {
          throw new Error(`Stored "${testCode}" but retrieved "${retrieved}" — local
        }
        ctx.codeFlowOk = true;
        return `Invite code stored and retrieved correctly ✓`;
      },
    },
    {
      name: "Invalid coach code is handled gracefully",
      run: async () => {
        // Verify Coach entity lookup with a bogus code returns empty, not an error
        let result;
        try {
          result = await base44.entities.Coach.filter({ invite_code: "__INVALID_HC_C
        } catch (err) {
```

```
          throw new Error(
            `Coach.filter() threw on unknown invite_code: ${err.message} — ` +
            "CoachInviteLanding will crash if code is not found"
          );
        }
        if (!Array.isArray(result)) throw new Error("Coach.filter() returned non-arr
        if (result.length > 0) throw new Error("Bogus invite code returned a match —
        return `Unknown invite_code correctly returns empty array ✓`;
      },
    },
  ],
},

{
  id: "coach_signup_functions",
  name: "Coach signup — account creation path",
  icon: "✍️",
  description: "Auth register and role assignment are functional for coach account t
  steps: [
    {
      name: "auth.register is callable (same as parent signup)",
      run: async () => {
        if (typeof base44.auth?.register !== "function") {
          throw new Error("base44.auth.register not available — coach signup page wi
        }
        return "base44.auth.register available ✓";
      },
    },
    {
      name: "Coach entity create/delete cycle",
      run: async (ctx) => {
        const testCoach = await base44.entities.Coach.create({
          name: "__healthcheck_coach__",
          school_or_org: "Health Check HS",
          sport: "Football",
          invite_code: `HC-TEST-${Date.now()}`,
          account_id: "hc_test_account",
          active: true,
        });
        if (!testCoach?.id) throw new Error("Coach.create() returned no id — entity
        ctx.testCoachId = testCoach.id;
        ctx.testInviteCode = testCoach.invite_code;
        return `Coach record created — id=${testCoach.id} code=${testCoach.invite_co
      },
```

```
    },
    {
      name: "Coach record readable by invite_code",
      run: async (ctx) => {
        const found = await base44.entities.Coach.filter({ invite_code: ctx.testInvi
        if (!Array.isArray(found) || found.length === 0) {
          throw new Error(`Coach not found by invite_code="${ctx.testInviteCode}" —
        }
        return `Coach found by invite_code ✓`;
      },
    },
    {
      name: "Cleanup — delete test coach record",
      run: async (ctx) => {
        if (ctx.testCoachId) {
          await base44.entities.Coach.delete(ctx.testCoachId);
        }
        return "Test Coach record deleted ✓";
      },
    },
  ],
},
```

Phase 3 — CoachRoster linking via stripeWebhook (highest risk)

Goal: When a parent pays using a coach invite link, a CoachRoster record is created linking them to the coach. Parent entitlement is completely unaffected if the coach linking fails.

Files changed

`base44/functions/stripeWebhook/entry.ts`

- After successful entitlement creation, add coach roster block:

```typescript
```

```
// ── COACH ROSTER LINKING (isolated — must not affect entitlement) ──
const coachInviteCode = session.metadata?.coach_invite_code || "";
if (coachInviteCode && accountId && athleteId) {
  try {
    const coaches = await base44.asServiceRole.entities.Coach.filter({
      invite_code: coachInviteCode,
      active: true,
    });
    if (coaches && coaches.length > 0) {
      const coach = coaches[0];
      // Check for existing roster link to avoid duplicates
      const existingLink = await base44.asServiceRole.entities.CoachRoster.filter({
        coach_id: coach.id,
        athlete_id: athleteId,
      }).catch(() => []);
      if (!existingLink || existingLink.length === 0) {
        await base44.asServiceRole.entities.CoachRoster.create({
          coach_id: coach.id,
          athlete_id: athleteId,
          account_id: accountId,
          linked_at: new Date().toISOString(),
          source: "invite_link",
        });
        console.log("CoachRoster created — coach:", coach.id, "athlete:", athleteId)
      } else {
        console.log("CoachRoster already exists — skipping duplicate");
      }
    } else {
      console.warn("Coach not found for invite_code:", coachInviteCode, "— skipping
    }
  } catch (coachErr) {
    // CRITICAL: log but never rethrow — parent payment must not fail due to coach l
    console.error("CoachRoster linking failed (non-critical):", coachErr.message);
  }
}
```

Health check journeys for Phase 3

```javascript
```

```
{
  id: "coach_roster_schema",
  name: "CoachRoster entity — schema and lifecycle",
  icon: "👥",
  description: "CoachRoster entity is queryable, can be created and deleted, " +
    "and correctly links a coach to an athlete.",
  steps: [
    {
      name: "CoachRoster entity is queryable",
      run: async (ctx) => {
        let roster;
        try {
          roster = await base44.entities.CoachRoster.filter({});
        } catch (err) {
          throw new Error(
            `CoachRoster not readable: ${err?.message || err} — ` +
            "create the CoachRoster entity before deploying Phase 3"
          );
        }
        if (!Array.isArray(roster)) throw new Error("CoachRoster.filter() returned n
        ctx.rosterCount = roster.length;
        return `CoachRoster entity reachable — ${roster.length} existing links`;
      },
    },
    {
      name: "CoachRoster create/read/delete cycle",
      run: async (ctx) => {
        const me = await base44.auth.me();
        const testLink = await base44.entities.CoachRoster.create({
          coach_id: "__hc_coach__",
          athlete_id: "__hc_athlete__",
          account_id: me.id,
          linked_at: new Date().toISOString(),
          source: "invite_link",
        });
        if (!testLink?.id) throw new Error("CoachRoster.create() returned no id");
        ctx.testRosterId = testLink.id;
        return `CoachRoster record created — id=${testLink.id}`;
      },
    },
    {
      name: "CoachRoster queryable by coach_id",
      run: async (ctx) => {
```

```
        const found = await base44.entities.CoachRoster.filter({ coach_id: "__hc_coa
        if (!Array.isArray(found) || !found.find(r => r.id === ctx.testRosterId)) {
          throw new Error("CoachRoster not found by coach_id — dashboard query will
        }
        return `CoachRoster found by coach_id ✓`;
      },
    },
    {
      name: "CoachRoster queryable by athlete_id",
      run: async (ctx) => {
        const found = await base44.entities.CoachRoster.filter({ athlete_id: "__hc_a
        if (!Array.isArray(found) || !found.find(r => r.id === ctx.testRosterId)) {
          throw new Error("CoachRoster not found by athlete_id — multi-coach parent
        }
        return `CoachRoster found by athlete_id ✓`;
      },
    },
    {
      name: "Cleanup — delete test roster link",
      run: async (ctx) => {
        if (ctx.testRosterId) await base44.entities.CoachRoster.delete(ctx.testRoste
        return "Test CoachRoster deleted ✓";
      },
    },
  ],
},

{
  id: "stripe_webhook_coach_isolation",
  name: "stripeWebhook — coach linking does not affect entitlement",
  icon: "💳",
  description: "Validates that the stripeWebhook coach linking block is isolated " +
    "from entitlement creation. Tests the data contract, not the webhook itself " +
    "(webhook is Stripe-triggered and cannot be called directly in tests).",
  steps: [
    {
      name: "Entitlement entity still writable (smoke test)",
      run: async (ctx) => {
        const me = await base44.auth.me();
        // Just verify the entity is reachable — don't create a real entitlement
        const ents = await base44.entities.Entitlement.filter({ account_id: me.id })
        if (!Array.isArray(ents)) throw new Error("Entitlement.filter() returned non
        ctx.existingEntitlements = ents;
        return `Entitlement entity reachable — ${ents.length} records for admin acco
```

```
    },
  },
  {
    name: "Coach lookup with unknown code returns empty (not an error)",
    run: async () => {
      // This mirrors exactly what stripeWebhook does when code is invalid
      let coaches;
      try {
        coaches = await base44.asServiceRole
          ? await base44.entities.Coach.filter({ invite_code: "__BAD_CODE__", acti
          : await base44.entities.Coach.filter({ invite_code: "__BAD_CODE__", acti
      } catch (err) {
        throw new Error(
          `Coach.filter() threw for unknown invite_code: ${err.message} — ` +
          "stripeWebhook would catch this, but entitlement creation is already don
          "Verify the catch block logs and continues."
        );
      }
      if (!Array.isArray(coaches)) throw new Error("Coach.filter() returned non-ar
      return `Unknown invite_code returns empty array — webhook catch block handle
    },
  },
  {
    name: "CoachRoster duplicate check works",
    run: async (ctx) => {
      // Create a roster link and verify a second filter for same pair returns it
      // (so the webhook dedup logic can find it and skip creating a duplicate)
      const link = await base44.entities.CoachRoster.create({
        coach_id: "__hc_dedup_coach__",
        athlete_id: "__hc_dedup_athlete__",
        account_id: "hc_test",
        linked_at: new Date().toISOString(),
        source: "invite_link",
      });
      ctx.dedupId = link.id;
      const existing = await base44.entities.CoachRoster.filter({
        coach_id: "__hc_dedup_coach__",
        athlete_id: "__hc_dedup_athlete__",
      });
      if (!Array.isArray(existing) || existing.length === 0) {
        throw new Error("Dedup filter returned empty — webhook will create duplica
      }
      return `Dedup filter found ${existing.length} existing link(s) — webhook wil
    },
```

```
    },
    {
      name: "Cleanup — delete dedup test record",
      run: async (ctx) => {
        if (ctx.dedupId) await base44.entities.CoachRoster.delete(ctx.dedupId);
        return "Dedup test record deleted ✓";
      },
    },
  ],
},
```

Phase 4 — Coach dashboard

Goal: Coaches can log in, see their roster, and view camp activity per player. Read-only. No writes to existing entities.

Files changed

`src/pages/CoachDashboard.jsx` ← new file

- Wrapped in `<CoachRoute>`
- Loads `Coach` record for current `account_id`
- Loads `CoachRoster` filtered by `coach_id`
- For each roster athlete, joins to `AthleteProfile` and `CampIntent`
- **Filters roster to athletes with active `Entitlement` only** — demo users excluded
- Two views: by player (default) and by camp

Health check journey for Phase 4

```javascript
```

```
{
  id: "coach_dashboard_data",
  name: "Coach dashboard — roster + camp data joins",
  icon: "📊",
  description: "Verifies the three-way join CoachRoster → AthleteProfile → CampInten
    "that powers the coach dashboard. Uses test data, cleaned up after.",
  steps: [
    {
      name: "AthleteProfile queryable by account_id (roster join)",
      run: async (ctx) => {
        const me = await base44.auth.me();
        const profiles = await base44.entities.AthleteProfile.filter({ account_id: m
        if (!Array.isArray(profiles)) throw new Error("AthleteProfile.filter() non-a
        ctx.adminProfiles = profiles;
        return `${profiles.length} AthleteProfile(s) found for admin account`;
      },
    },
    {
      name: "CampIntent queryable by athlete_id",
      run: async (ctx) => {
        if (ctx.adminProfiles.length === 0) {
          return "No athlete profiles for admin — CampIntent join skipped (expected
        }
        const athleteId = ctx.adminProfiles[0].id;
        const intents = await base44.entities.CampIntent.filter({ athlete_id: athlet
        if (!Array.isArray(intents)) throw new Error("CampIntent.filter() non-array
        return `CampIntent queryable by athlete_id — ${intents.length} intent(s) fou
      },
    },
    {
      name: "Entitlement filter for active paid athletes (demo exclusion)",
      run: async () => {
        const active = await base44.entities.Entitlement.filter({ status: "active" }
        if (!Array.isArray(active)) throw new Error("Entitlement.filter() non-array
        const accountIds = new Set(active.map(e => e.account_id).filter(Boolean));
        return `${accountIds.size} unique paid account_ids — coach will only see the
      },
    },
  ],
},
```

Phase 5 — Messaging + parent inbox + email notification

Goal: Coaches can send camp recommendations. Parents see them in their HQ. Email fires on send. Parent can tap one of three response buttons.

Files changed

`base44/functions/sendCoachMessage/entry.ts` ← new function

- Auth: caller must have `role === "coach"`
- Creates `CoachMessage` record per athlete (one record per athlete-camp-send)
- Calls email send (wrapped in try/catch — message saves even if email fails)
- Returns `{ ok: true, sent: N }`

`src/pages/Workspace.jsx`

- Add `CoachMessages` section — conditionally rendered only when messages exist
- Loads `CoachMessage.filter({ athlete_id: currentAthleteId })`
- Each message shows: coach name, message body, camp card (name + date + location)
- Three response buttons: 👀 Reviewing / ✅ Registered / ✕ Won't attend
- On button tap: `CoachMessage.update(id, { status: "reviewing" | "registered" | "not_attending" })`

`src/pages/CoachDashboard.jsx` — add to existing page

- Camp browse section (reuses existing camp data)
- "Recommend to players" action on any camp
- Message history tab with per-player status dots

Health check journeys for Phase 5

```javascript
```

```
{
  id: "coach_message_lifecycle",
  name: "CoachMessage — full send and response lifecycle",
  icon: "💬",
  description: "CoachMessage entity supports create, status update through all three
    "response states, and auto-status flip when CampIntent registered.",
  steps: [
    {
      name: "CoachMessage entity is queryable",
      run: async (ctx) => {
        let msgs;
        try {
          msgs = await base44.entities.CoachMessage.filter({});
        } catch (err) {
          throw new Error(
            `CoachMessage not readable: ${err?.message || err} — ` +
            "create the CoachMessage entity before deploying Phase 5"
          );
        }
        if (!Array.isArray(msgs)) throw new Error("CoachMessage.filter() returned no
        return `CoachMessage entity reachable — ${msgs.length} existing messages`;
      },
    },
    {
      name: "CoachMessage create with status=pending",
      run: async (ctx) => {
        const me = await base44.auth.me();
        const msg = await base44.entities.CoachMessage.create({
          coach_id: "__hc_coach__",
          athlete_id: "__hc_athlete__",
          camp_id: "__hc_camp__",
          body: "[HEALTHCHECK] automated test message — safe to ignore",
          sent_at: new Date().toISOString(),
          status: "pending",
          account_id: me.id,
        });
        if (!msg?.id) throw new Error("CoachMessage.create() returned no id");
        ctx.testMsgId = msg.id;
        return `CoachMessage created — id=${msg.id} status=pending`;
      },
    },
    {
      name: "Status update → reviewing",
```

```
    run: async (ctx) => {
      await base44.entities.CoachMessage.update(ctx.testMsgId, {
        status: "reviewing",
        opened_at: new Date().toISOString(),
      });
      const updated = await base44.entities.CoachMessage.filter({ id: ctx.testMsgI
      const msg = Array.isArray(updated) ? updated.find(m => m.id === ctx.testMsgI
      if (!msg) throw new Error("CoachMessage not found after update");
      if (msg.status !== "reviewing") throw new Error(`Expected status=reviewing,
      return `Status updated to reviewing ✓`;
    },
  },
  {
    name: "Status update → registered",
    run: async (ctx) => {
      await base44.entities.CoachMessage.update(ctx.testMsgId, { status: "register
      const updated = await base44.entities.CoachMessage.filter({ id: ctx.testMsgI
      const msg = Array.isArray(updated) ? updated.find(m => m.id === ctx.testMsgI
      if (!msg) throw new Error("CoachMessage not found after update");
      if (msg.status !== "registered") throw new Error(`Expected status=registered
      return `Status updated to registered ✓`;
    },
  },
  {
    name: "Status update → not_attending",
    run: async (ctx) => {
      await base44.entities.CoachMessage.update(ctx.testMsgId, { status: "not_atte
      const updated = await base44.entities.CoachMessage.filter({ id: ctx.testMsgI
      const msg = Array.isArray(updated) ? updated.find(m => m.id === ctx.testMsgI
      if (!msg) throw new Error("CoachMessage not found after update");
      if (msg.status !== "not_attending") throw new Error(`Expected status=not_att
      return `Status updated to not_attending ✓`;
    },
  },
  {
    name: "CoachMessage queryable by athlete_id (parent inbox query)",
    run: async (ctx) => {
      const found = await base44.entities.CoachMessage.filter({ athlete_id: "__hc_
      if (!Array.isArray(found) || !found.find(m => m.id === ctx.testMsgId)) {
        throw new Error("CoachMessage not found by athlete_id — parent inbox will
      }
      return `CoachMessage found by athlete_id — parent inbox query works ✓`;
    },
  },
```

```
    {
      name: "CoachMessage queryable by coach_id (coach message tracker query)",
      run: async (ctx) => {
        const found = await base44.entities.CoachMessage.filter({ coach_id: "__hc_co
        if (!Array.isArray(found) || !found.find(m => m.id === ctx.testMsgId)) {
          throw new Error("CoachMessage not found by coach_id — coach message tracke
        }
        return `CoachMessage found by coach_id — coach tracker query works ✓`;
      },
    },
    {
      name: "Cleanup — delete test message",
      run: async (ctx) => {
        if (ctx.testMsgId) await base44.entities.CoachMessage.delete(ctx.testMsgId);
        return "Test CoachMessage deleted ✓";
      },
    },
  ],
},

{
  id: "coach_message_autostatus",
  name: "CoachMessage auto-status — CampIntent registered flip",
  icon: "🔄",
  description: "When a CampIntent is set to registered for the same athlete+camp " +
    "as a CoachMessage, the CoachMessage status should auto-flip to registered. " +
    "Validates the query pattern needed to implement this.",
  steps: [
    {
      name: "CampIntent filterable by athlete_id + camp_id",
      run: async () => {
        // This is the query the auto-flip logic uses to detect registration
        const intents = await base44.entities.CampIntent.filter({
          athlete_id: "__hc_probe_athlete__",
          camp_id: "__hc_probe_camp__",
        });
        if (!Array.isArray(intents)) {
          throw new Error(
            "CampIntent.filter({ athlete_id, camp_id }) non-array — " +
            "auto-status flip cannot detect registration"
          );
        }
        // Empty is expected — probe IDs don't exist
        return `CampIntent filterable by athlete_id + camp_id ✓ (${intents.length} m
```

```
      },
    },
    {
      name: "CoachMessage filterable by athlete_id + camp_id",
      run: async () => {
        // This is the query to find which CoachMessages to update on registration
        const msgs = await base44.entities.CoachMessage.filter({
          athlete_id: "__hc_probe_athlete__",
          camp_id: "__hc_probe_camp__",
        });
        if (!Array.isArray(msgs)) {
          throw new Error(
            "CoachMessage.filter({ athlete_id, camp_id }) non-array — " +
            "auto-status flip cannot find the message to update"
          );
        }
        return `CoachMessage filterable by athlete_id + camp_id ✓`;
      },
    },
  ],
},

{
  id: "coach_email_notification",
  name: "Coach message email — send function reachable",
  icon: "📧",
  description: "The sendCoachMessage function exists and is invokable. " +
    "Email delivery is non-critical — message saves even if email fails.",
  steps: [
    {
      name: "sendCoachMessage function is registered",
      run: async () => {
        // Invoke with a dry_run flag — function should return ok:false with a
        // 'dry_run' or 'validation' error rather than crashing
        let res;
        try {
          res = await base44.functions.invoke("sendCoachMessage", {
            dry_run: true,
            coach_id: "__hc__",
            athlete_ids: [],
            camp_id: "__hc__",
            body: "[HEALTHCHECK]",
          });
        } catch (err) {
```

```
          throw new Error(
            `sendCoachMessage function not reachable: ${err?.message || err} — ` +
            "deploy the function before testing Phase 5"
          );
        }
        const data = res?.data;
        if (data === undefined) throw new Error("sendCoachMessage returned undefined
        return `sendCoachMessage reachable — response: ok=${data?.ok} ${data?.error
      },
    },
    {
      name: "Email function available for notification sends",
      run: async () => {
        // Check that whatever email function is used (sendHealthAlert or similar) i
        // Coach email uses same infrastructure as existing system emails
        let res;
        try {
          res = await base44.functions.invoke("sendHealthAlert", {
            toEmail: "healthcheck-probe@example.com",
            failures: [],
            runDate: new Date().toLocaleDateString(),
          });
        } catch (err) {
          throw new Error(
            `Email send function not reachable: ${err?.message || err} — ` +
            "coach notification emails will silently fail"
          );
        }
        return `Email infrastructure reachable ✓ (probe send to non-real address)`;
      },
    },
  ],
},
```

Phase 6 — Admin stats expansion (ProductMetrics)

Goal: ProductMetrics shows registered parents, athletes, coaches, favorited/registered counts, and coach channel conversion. No new entities — all derived from existing data plus the new coach entities.

Files changed

`src/pages/ProductMetrics.jsx`

- Add `athletes` data fetch: `AthleteProfile.filter({ active: true })`
- Add `coaches` data fetch: `Coach.filter({ active: true })` — shows 0 until Phase 2 deploys
- Add `coachRosters` fetch for conversion metric
- New "Users" section at top with stat cards: registered parents, total athletes, registered coaches, athletes per account
- Add to Engagement section: favorite→registered rate, athletes with zero camps, top 10 favorited camps, top 10 registered camps
- New "Coach Channel" section: total coaches, roster links, messages sent, response rate, coach→paid conversion — hidden if no coaches exist

`src/pages/AdminOps.jsx`

- Add "Coach Network" card to the existing tools grid linking to `/CoachNetworkAdmin`

`src/pages/CoachNetworkAdmin.jsx` ← new file, wrapped in `<AdminRoute>`

- Table of all coaches with roster size, messages sent, invite link click count, paid conversion
- Can deactivate a coach (set `active: false` on Coach record)

Health check journey for Phase 6

```javascript
```

```
{
  id: "admin_stats_data",
  name: "Admin stats — ProductMetrics data sources",
  icon: "📈",
  description: "All entities and queries needed for the expanded ProductMetrics " +
    "stats section are reachable and return expected shapes.",
  steps: [
    {
      name: "AthleteProfile total count queryable",
      run: async (ctx) => {
        const profiles = await base44.entities.AtheleteProfile.filter({ active: true
        if (!Array.isArray(profiles)) throw new Error("AtheleteProfile.filter({active
        ctx.totalAthletes = profiles.length;
        return `${profiles.length} active AtheleteProfiles — total athletes stat read
      },
    },
    {
      name: "CampIntent favorite/registered counts queryable",
      run: async (ctx) => {
        const favorites = await base44.entities.CampIntent.filter({ status: "favorit
        const registered = await base44.entities.CampIntent.filter({ status: "regist
        if (!Array.isArray(favorites)) throw new Error("CampIntent favorite filter n
        if (!Array.isArray(registered)) throw new Error("CampIntent registered filte
        ctx.totalFavorites = favorites.length;
        ctx.totalRegistered = registered.length;
        const rate = favorites.length > 0 ? ((registered.length / favorites.length)
        return `${favorites.length} favorites, ${registered.length} registered — con
      },
    },
    {
      name: "Top favorited camps computation (groupBy camp_id)",
      run: async () => {
        const intents = await base44.entities.CampIntent.filter({});
        if (!Array.isArray(intents)) throw new Error("CampIntent.filter() non-array
        const bycamp = {};
        for (const i of intents) {
          if (i.camp_id) bycamp[i.camp_id] = (bycamp[i.camp_id] || 0) + 1;
        }
        const topCamps = Object.entries(bycamp)
          .sort((a, b) => b[1] - a[1])
          .slice(0, 10);
        return `Top camps computation works — ${topCamps.length} camps in top-10 lis
      },
```

```
    },
    {
      name: "Coach entity queryable for admin stats",
      run: async (ctx) => {
        let coaches;
        try {
          coaches = await base44.entities.Coach.filter({});
        } catch (err) {
          // Coach entity may not exist yet in Phase 1 — warn, don't fail
          return `⚠ Coach entity not yet available: ${err?.message} — coach stats wi
        }
        if (!Array.isArray(coaches)) throw new Error("Coach.filter() non-array");
        return `${coaches.length} coach(es) — admin stats coach section ready ✓`;
      },
    },
    {
      name: "Athletes with zero camps — derivable from existing data",
      run: async (ctx) => {
        if (!ctx.totalAthletes) return "AthleteProfile count not available — run pre
        const intents = await base44.entities.CampIntent.filter({});
        if (!Array.isArray(intents)) throw new Error("CampIntent.filter() non-array"
        const athletesWithCamps = new Set(intents.map(i => i.athlete_id).filter(Bool
        const withZero = ctx.totalAthletes - athletesWithCamps;
        return `${athletesWithCamps} athletes with camps, ${withZero} with zero — st
      },
    },
  ],
},
```

Deployment sequence

```
Phase 1  →  Deploy  →  Run health check (coach_role_bypass)
Phase 2  →  Deploy  →  Run health check (coach_invite_code_flow,
coach_signup_functions, coach_entity_schema)
Phase 3  →  Deploy  →  Run health check (coach_roster_schema,
stripe_webhook_coach_isolation)
           ↳ Also re-run existing "Subscriber — Entitlement Check" journey to
confirm no regression
Phase 4  →  Deploy  →  Run health check (coach_dashboard_data)
Phase 5  →  Deploy  →  Run health check (coach_message_lifecycle,
coach_message_autostatus, coach_email_notification)
```

```
Phase 6 → Deploy → Run health check (admin_stats_data)
          ↳ Also re-run full health check suite to confirm no regressions
```

New entities summary

Entity	Fields	Created in phase
`Coach`	id, account_id, name, school_or_org, sport, invite_code, active, created_at	Phase 2
`CoachRoster`	id, coach_id, athlete_id, account_id, linked_at, source	Phase 3
`CoachMessage`	id, coach_id, athlete_id, camp_id, body, sent_at, opened_at, status, account_id	Phase 5

Existing entities — additions only

Entity	Addition	Phase
Account (auth user)	`role: "parent" | "coach" | "admin"`	Phase 1
Stripe session metadata	`coach_invite_code` field passed through checkout	Phase 3

Files changed summary

File	Change type	Phase
`useSeasonAccess`	Edit — add coach role bypass	1
`AuthRedirect.jsx`	Edit — add coach routing branch	1
`CoachRoute.jsx`	New file	1
`pages.config.js`	Edit — add new routes	1
`Home.jsx`	Edit — add coach signup link	2
`CoachSignup.jsx`	New file	2

File	Change type	Phase
`CoachInviteLanding.jsx`	New file	2
`Checkout.jsx`	Edit — read + pass coach_invite_code	3
`stripeWebhook/entry.ts`	Edit — add coach roster block	3
`CoachDashboard.jsx`	New file	4
`sendCoachMessage/entry.ts`	New function	5
`Workspace.jsx`	Edit — add coach messages section	5
`ProductMetrics.jsx`	Edit — add Users + Coach Channel sections	6
`AdminOps.jsx`	Edit — add Coach Network card	6
`CoachNetworkAdmin.jsx`	New file	6
`AppHealthCheck.jsx`	Edit — add Coach Feature journey group	Each phase